KTF: A Framework for Semantic Knowledge Transformation

Keywords: Knowledge Engineering, Knowledge Transformation, Ontologies, Asset Administration Shells, Semantic Planning Knowledge, Software Framework

Abstract: Semantic knowledge artifacts often need to be transformed between heterogeneous representations, tool-specific formats, and deployment contexts while preserving their semantic meaning. This paper presents KTF, the Knowledge Transformation Framework, an open-source framework for transforming, validating, composing, and manipulating knowledge artifacts. KTF is implemented as a collection of tools and currently focuses on semantic planning knowledge. Its planning domain transformation tool connects Plato, an ontology schema for representing planning domains, Pallas, a representation of planning knowledge using Asset Administration Shells as standardized digital descriptions of industrial assets, and Planning Domain Definition Language artifacts, which are used by AI planners to describe planning domains and problems. The framework further includes a PlanInterpreter for converting planner outputs into structured formats such as JSON and BPMN, and a JSOntoConverter for validating and converting JSOnto, a JSON-based ontology syntax, into established RDF/OWL serializations. KTF can be used locally through a command line, as a Maven dependency, or through a Dockerized REST API. Its application in connected evaluations and a distributed edge-cloud production scenario provides initial evidence that the framework can support practical knowledge-engineering workflows across semantic modeling, planning, validation, and execution.

1 INTRODUCTION

There is no such thing as the perfect semantic data structure, the perfect model, the perfect syntax, or the perfect file format. Different use cases, tooling, or even preferences among users can require specialized representations. One might even resort to using multiple representations for different purposes. For example, in Artificial Intelligence (AI) planning, the de facto standard language for representing planning models is the Planning Domain Definition Language (PDDL). However, due to difficulties in modeling complex domains in this language, it is a common approach to model the semantic planning information in other structures that are easier to model in, such as ontologies, and then later transform them into PDDL (Malburg et al., 2023; Klusch et al., 2005; Wickler et al., 2014). Similarly, an ontology may be maintained in a human-oriented syntax, such as Turtle (Beckett et al., 2014), but exchanged with tools that expect more established Resource Description Framework (RDF) or Web Ontology Language (OWL) serializations. These situations require more than file-format conversion. They require transformation tools that preserve relevant semantics, adapt knowledge to target systems, and support composition across heterogeneous representations.

For this purpose, this paper presents the Knowledge Transformation Framework (KTF). KTF is an open-source framework, available under the MIT license on GitHub[1], that supports such transformations and other operations, such as validation, through a set of integrated but expandable tools. Currently, the framework is heavily focused on semantic planning information. It provides tools such as the *PlanningDomainTransformer*, which transforms planning domains between different representations, and the *PlanInterpreter*, which converts plans between different output formats. In this paper, the term *transformer* is used in its general sense of transforming information between structures and does not refer to the transformer architecture known from machine learning. The framework also contains tools for non-planning use-cases, such as the *JSOntoConverter*, which can transform between the *JSOnto* (JSOnto, 2026) ontology syntax and other more established syntaxes, such as Turtle. These tools can be used locally through a command-line interface or remotely through a Representational State Transfer (REST) Application Programming Interface (API), and the framework is also distributed through a Docker image. Additionally, the framework can be imported as a Apache Maven[2] dependency to use in other projects where the tools can also be extended through implementing ready-to-use Java interfaces.

[1] https://github.com/user140613/KTF

[2] https://maven.apache.org/

The main technical contribution of KTF lies in providing tools that can make existing semantic knowledge structures operational. For example, KTF can be used to convert between different semantic models for providing semantic planning knowledge, such as Plato or Pallas. Plato was introduced in previous work as an ontology schema for semantic planning knowledge (Plato, 2026) . Pallas provides an Asset Administration Shell (AAS)-based counterpart for representing similar planning knowledge in Industry 4.0 (I4.0) environments (Pallas, 2026; Industrial Digital Twin Association and Plattform Industrie 4.0, 2024) . An AAS is a semantic data structure for a standardized digital representation of an industrial asset that structures and exchanges asset-related information across systems (see Section 2.2). Another semantic technology being operationalized by KTF is JSOnto. JSOnto provides a JavaScript Object Notation (JSON)-based ontology syntax intended for human-friendly ontology engineering (JSOnto, 2026) . KTF connects these artifacts; for example, it can transform between Plato, Pallas, and PDDL or validate and convert JSON to ontologies.

At the same time, KTF is not only a set of converters. Its PlanningDomainTransformer can also adapt generated planning models to the requirements of specific planning systems and use cases. In AI planning, a model describes which actions are possible, under which conditions they can be applied, and how they change the current situation. However, different planners support different parts of PDDL, especially when conditions, numeric values, or arithmetic expressions are involved. KTF therefore provides configurable manipulation steps that can simplify or rewrite planning models before they are given to a planner. For example, it can replace conditions that require something not to be true by equivalent explicitly modeled predicates, precompute certain numeric expressions, update a planning problem after some actions have already been executed, or discretize bounded numeric values. These transformations help reuse the same semantic source model with planners that have different technical capabilities.

The remainder of this paper is organized as follows. Section 2 positions KTF in relation to compactly selected related tools. Section 3 presents the actual framework, including its architecture and tools. Section 4 reports the application-based evaluation. Section 5 discusses implications and limitations, and Section 6 concludes the paper.

2 FOUNDATIONS AND RELATED WORK

Since KTF is currently heavily focused on semantic planning knowledge, this section summarizes the required planning background in Section 2.1. Additionally, a brief introduction to AASs is given in Section 2.2. Section 2.3 then introduces directly related semantic artifacts and positions KTF relative to selected related software frameworks and tools.

2.1 AI Planning and PDDL

AI planning addresses the problem of finding a sequence of actions that transforms a system from a given initial state to a desired goal (Green, 1969; Haslum, 2006). In classical planning, states capture the relevant properties of the world model, while actions specify possible state transitions by means of applicability conditions and effects. A planning problem is defined by an initial state, a goal condition, and a set of available actions. A solution is typically a finite sequence of parameterized actions whose execution leads to a state satisfying the goal.

PDDL is the de facto standard language used to describe planning domains and planning problems (Haslum et al., 2019; Helmert, 2009; Ammar and Bhiri, 2018). A domain file contains reusable domain-level definitions, including requirements, types, predicates, functions, constants, and actions. A problem file instantiates such a domain by specifying the objects, the initial state, the goal, and optionally an optimization metric. Extensions of PDDL further support temporal and numeric modeling constructs, such as durative actions, conditions and effects with temporal scope, and arithmetic expressions for modeling costs, durations, and capacities (Fox and Long, 2003).

The degree to which planners support different PDDL fragments differs considerably. A domain that is valid for one planner may fail with another planner, particularly when temporal constructs, numeric expressions, or more complex logical structures are used. As a consequence, maintaining knowledge directly in PDDL tightly links domain modeling to the syntax and capabilities of a specific planner. KTF operationalizes the rationale of semantic source models by allowing the maintenance of planning knowledge in, for example, ontologies and compiling it into planner-specific PDDL artifacts.

2.2 Asset Administration Shell

Asset Administration Shells provide standardized digital representations of assets in I4.0 environments

(Industrial Digital Twin Association and Plattform Industrie 4.0, 2024), where AI, data analytics, the Internet of Things (IoT), and distributed systems are integrated into industrial workflows (Gilchrist, 2016). They can describe both physical and non-physical assets and structure the information associated with them. The information in an AAS is organized through standardized submodels. Each submodel can contain other a set of elements: *Properties* capture scalar values, such as names, identifiers, parameters, and numeric values. *Reference Elements* refer to other elements and may also point across different AASs. *Submodel Collections (SMCs)* are used to group related information into structured elements, whereas *Submodel Lists (SMLs)* represent homogeneous lists. Together, these elements support the representation of asset-related information at different levels of granularity, ranging from individual values to nested and structured descriptions.

2.3 Related Work

KTF is positioned at the intersection of ontology tooling, model transformation, and planning-domain engineering. The following discussion is deliberately compact and does not aim to survey these areas. Instead, it introduces the semantic artifacts directly connected by KTF and identifies software frameworks and transformation approaches most relevant for positioning KTF.

KTF builds on three directly related knowledge-engineering artifacts. Plato is an ontology schema for semantic planning knowledge (Plato, 2026) . It represents planning-domain concepts, such as types, objects, predicates, actions, costs, and metrics as ontology elements instead of requiring this knowledge to be maintained directly in PDDL. Pallas provides an AAS-based counterpart for representing similar planning knowledge in I4.0 and other AAS-centered environments (Pallas, 2026; Industrial Digital Twin Association and Plattform Industrie 4.0, 2024) . It uses AAS submodels and submodel elements to represent planning constructs and supports the distribution of planning knowledge across multiple asset representations. JSOnto addresses a different but related problem: it is a modular JSON-based ontology syntax intended to make ontology engineering more human-friendly and easier to integrate into software-engineering workflows (JSOnto, 2026) . These three artifacts motivate the central role of KTF: Plato and Pallas provide semantic source models for planning knowledge, while JSOnto provides an alternative syntax for authoring and maintaining ontologies.

Ontology frameworks such as the OWL API (Horridge and Bechhofer, 2011) and Apache Jena (Carroll et al., 2004) provide mature programmatic support for parsing, serializing, querying, and manipulating RDF and OWL artifacts. ROBOT extends this tool-oriented perspective by supporting reproducible ontology workflows such as conversion, merging, querying, verification, reporting, and repair (Jackson et al., 2019). These systems are close to KTF's JSOnto-related functionality because they demonstrate that ontology engineering requires robust tooling beyond editing and serialization. KTF differs by combining ontology-syntax conversion with semantic planning-domain transformation and plan-output interpretation in one framework.

The Consumer–Intermediary–Producer (CIP) methodology used in KTF's PlanningDomainTransformer (see Section 3.2) is related to model-transformation and pivot-model architectures. Atlas Transformation Language (ATL) is a mature model-transformation language and tooling environment (Jouault et al., 2008). Pivot representations have also been used to separate language-independent rewriting from source and target languages, for example in work on rewriting constraint models with metamodels (Chenouard et al., 2010). In AI planning, intermediate representations also play a central role. Fast Downward, for instance, translates PDDL tasks into multi-valued planning tasks before search (Helmert, 2011). These systems do not address the same knowledge-engineering problem as KTF, but they support the general design idea that intermediate representations can be more than implementation conveniences.

Planning knowledge engineering tools form the closest planning-specific comparison. itSIMPLE supports planning-domain engineering with Unified Modeling Language (UML)-based models that can be translated to PDDL (Vaquero et al., 2013). Graphical Interface for Planning with Objects (GIPO) provides object-centered support for planning-domain modeling and validation (McCluskey et al., 2003). The Unified Planning library offers a modern API for defining planning problems, selecting engines, validating plans, simulating plans, and compiling or reformulating planning tasks (Micheli et al., 2025). VAL is a well-established tool for validating PDDL plans (Howey et al., 2004). KTF shares the view that planning artifacts should be generated, validated, transformed, or interpreted through tooling, but its focus is different: KTF connects semantic source models such as Plato and Pallas with planner-oriented artifacts by, for example, allowing the conversion of semantic planning knowledge stored in these models into a planning system-specific PDDL problem description.

Additionally, it combines this with ontology-syntax transformation and service-oriented deployment.

3 THE KNOWLEDGE TRANSFORMATION FRAMEWORK

The KTF is an open-source framework for transforming, validating, composing, and manipulating knowledge artifacts. Rather than being a single monolithic artifact, KTF is implemented as a collection of tools that can be used independently or combined into larger transformation workflows. This section presents KTF from the perspective of its tool structure. Section 3.1 first describes the overall architecture and usage modes. Section 3.2 then introduces the PlanningDomainTransformer. Section 3.3 describes the PlanInterpreter, and Section 3.4 describes the JSOntoConverter.

3.1 Architecture and Usage

KTF is organized as a set of tools that share common access and deployment mechanisms. This tool-oriented organization allows KTF to support both isolated conversions and multi-step knowledge-engineering workflows. For example, a Plato ontology may be modeled in JSOnto to benefit from JSOnto's modular JSON-based syntax and later be converted into PDDL. However, the PlanningDomainTransformer currently consumes OWL/RDF-based Plato ontologies with Apache Jena and does not process JSOnto files directly. Such a workflow would therefore first validate the JSOnto ontology and convert it into an RDF/Extensible Markup Language (XML) serialization with the JSOntoConverter. The resulting ontology can then be processed by the PlanningDomainTransformer to manipulate and validate the planning domain and generate PDDL artifacts. After a planner has then generated a plan file, it can be transformed into a better structure and file format using the PlanInterpreter.

KTF can be used in three main ways. First, users can execute the framework locally through the command line, which is suitable for experiments, reproducible examples, and integration into scripts. Second, KTF can be deployed as a Docker image that exposes the tools through an Hypertext Transfer Protocol (HTTP) REST API. In the standard configuration, the API is reachable on port 5000 and provides Swagger-based documentation at the base URL. Third, KTF can be imported as a Java dependency in Maven-based projects. This allows applications to call selected classes and functions directly and, for example, to reuse planning-domain manipulation functionality from the PlanningDomainTransformer without using the whole transformation process. Additionally, a Maven import allows an application to extend KTF's functionality. For example, by implementing a new consumer for the PlanningDomainTransformer using the provided *Consumer* interface.

3.2 PlanningDomainTransformer

The PlanningDomainTransformer is the largest and most planning-specific component of KTF. It transforms semantic planning information between different data structures and schemas. The internal design of the PlanningDomainTransformer follows the CIP methodology. Consumers read source-specific representations and translate them into a generic intermediate representation of a planning domain. Producers then generate target artifacts from this intermediate representation. Figure 1 illustrates this structure. The central advantage of this design is that source-specific parsing, representation-independent manipulation, and target-specific generation are separated. New source or target representations can be connected by implementing the generic consumer or producer interfaces, rather than by implementing a direct converter for every existing source-target pair.

The current implementation supports consumers for Plato ontologies, Pallas AAS models, and PDDL domains. On the output side, it supports producers for PDDL, Pallas, and planning-domain statistics. In this way, the tool connects ontology-based planning knowledge, AAS-based planning knowledge, and planner-oriented artifacts.

The PlanningDomainTransformer is configured through a JSON configuration file. The configuration specifies the consumer, producer, domain type, input and output paths, planning objects, initial state, goal state, metric, domain and problem names, and optional manipulation settings. This makes transformations reproducible and allows the same semantic source model to be transformed into different target artifacts by changing the configuration. For example, a Plato ontology or Pallas AAS model can be combined with different initial states, goal states, or planner-specific settings to produce different PDDL problem files.

The tool supports semantic planning-domain transformation between Plato, Pallas, and PDDL. Plato represents planning knowledge as ontology objects, including types, entities, predicates, actions, parameters, preconditions, effects, functions, costs,

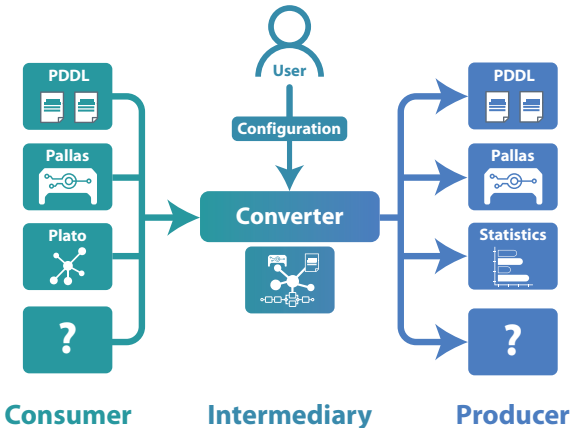

Figure 1: CIP methodology used inside the PlanningDomainTransformer. Consumers translate source structures into an intermediate planning-domain representation, and producers generate target artifacts.

metrics, and numeric constants (Plato, 2026) . Pallas represents comparable planning knowledge in AAS submodels and submodel elements, making it suitable for AAS-centered I4.0 environments (Pallas, 2026) . The PlanningDomainTransformer operationalizes these models by generating planner-oriented PDDL domain and problem files, or by generating Pallas models from supported semantic planning information.

Beyond conversion, the PlanningDomainTransformer includes manipulation options for adapting planning artifacts to target use cases and planner capabilities. Practical PDDL planners differ in their support for temporal, numeric, and logical constructs. KTF therefore provides options such as `removeNegativePreconditions` and `removeNumericExpressions`. The former substitutes negative preconditions by predicates that explicitly represent the negated statement, while the latter replaces numeric expressions with precomputed function calls where this is supported by the modeled knowledge and configuration. These options do not remove the need to understand the target planner, but they allow some planner-specific restrictions to be handled during transformation rather than manually in the source model.

The PlanningDomainTransformer also supports action-delay manipulation. The configuration option `delayActions` lists actions that should be delayed. For these actions, the transformer adds a precondition that requires another non-delayed action to be executed before them. The predicate used for this purpose is specified by `actionDelayPredicate`. Conversely, `removeActionDelay` can be used to remove delay-related preconditions and effects identified through that predicate. This provides a configurable mechanism for representing or removing simple ordering constraints at transformation time and can also be used to force the generation of alternative plans.

A further manipulation capability is domain forwarding by action replaying. The configuration option `forwardingSteps` contains a list of plan steps, each represented by a predicate and its arguments. These steps are replayed to forward the planning domain or problem to a later state after a number of actions have already been executed. This is useful when a semantic planning model is maintained as a reusable source artifact, but a concrete planning problem must reflect partial execution of a process. KTF does not replace a plan validator such as VAL; instead, replaying is used as a transformation operation within the generation pipeline.

KTF also supports automated numeric discretization for bounded numeric information. In industrial and production-planning settings, relevant properties often include values such as temperatures, quantities, capacities, costs, or durations. Plato and Pallas allow such values to be modeled with minimum values, maximum values, and step sizes (Plato, 2026; Pallas, 2026) . The PlanningDomainTransformer can use this information to generate discrete planning entities or values for planner-compatible artifacts. This should be understood as an engineering transformation for bounded numeric structures, not as a complete treatment of arbitrary numeric planning.

The StatisticsProducer can generate statistics about a planning domain. Such statistics are useful for inspecting generated domains, comparing versions, and detecting unexpected growth in the number of actions, predicates, or other planning constructs. This reinforces the role of the PlanningDomainTransformer as a domain engineering tool rather than only as a file generator.

Finally, the manipulation capabilities of the PlanningDomainTransformer are not limited to complete transformation workflows. When KTF is imported as a Maven dependency in another Java project, selected components can also be used programmatically. This allows applications to construct, inspect, adapt, or forward planning-domain representations directly, without invoking the command-line interface or REST API. In this usage mode, KTF can serve as a reusable library for planning-domain manipulation, for example when an external system already maintains its own transformation pipeline but requires KTF's functionality for planner-specific adaptation, action replaying, or numeric discretization.

3.3 PlanInterpreter

The PlanInterpreter complements the input-side transformation tools by processing outputs generated by PDDL planners. It converts planner output files into more commonly used structured formats such as JSON or Business Process Model and Notation (BPMN) models. This is useful in workflows where KTF first generates PDDL artifacts, an external planner computes a plan, and the resulting plan must then be passed to downstream applications or process-oriented tools. Just as the PlanningDomainTransformer the PlanInterpreter also uses the CIP methodology where each consumer consumes plan files from a certain planner (though multiple planners can share the same consumer if using the same structure for their plan files), and each producer produces a plan representation, such as BPMN or JSON.

The command-line interface for the PlanInterpreter expects a plan file, an input format, an output format, and an output path. The REST API exposes the same functionality, where the request contains the plan and may contain a configuration specifying input and output formats.

The PlanInterpreter should not be understood as a general plan validator. Its purpose is to transform planner output into formats that are easier to consume in subsequent systems. This makes it part of KTF's broader knowledge-transformation workflow: semantic planning knowledge can be transformed into PDDL, external planning can be applied, and the resulting plan can then be transformed again into a structured representation.

3.4 JSOntoConverter

The JSOnto Converter addresses ontology-syntax transformation. JSOnto is a JSON-based ontology syntax designed for human-friendly, modular, text-based ontology engineering (JSOnto, 2026) . Since standard ontology tools commonly expect established RDF or OWL serializations, KTF provides tooling for converting JSOnto ontologies into formats such as RDF/XML or Turtle.

In addition to conversion, the JSOntoConverter can validate JSOnto ontologies. Validation can be executed without specifying an output serialization, and validation results can optionally be written to a JSON file. The converter reports detailed information about encountered problems, for example, schema violations or invalid data types.

The JSOntoConverter is also used in composed KTF workflows. In particular, the `JSOntoToPDDLConverter` mode first converts a JSOnto ontology into an OWL/RDF representation and then calls the PlanningDomainTransformer to generate PDDL artifacts. This composition is relevant when a Plato-compatible ontology is authored in JSOnto. It allows users to benefit from JSOnto's modular JSON-based modeling style while still using the PlanningDomainTransformer, which currently works on the converted ontology representation rather than on JSOnto directly.

4 APPLICATION-BASED EVALUATION

KTF is a software framework, and no independent controlled user study or benchmark evaluation of KTF itself has yet been conducted. However, its use in connected research artifacts and practical research projects provides indirect evidence for feasibility, maturity, and reusability. This evaluation is therefore not intended to show superiority over alternative approaches. Instead, it documents how KTF has been used as an enabling transformation and tooling component in the evaluations of Plato and Pallas, in the operationalization of JSOnto, and in the [redacted for blind review] project.

KTF was an essential component in the evaluation of Plato. In that work, KTF transformed Plato ontologies into PDDL domain and problem files. The evaluation covered several example ontologies, including classical planning, temporal planning, production planning, and numeric discretization scenarios. The generated PDDL artifacts were then used with suitable planners, and the resulting plans were validated with VAL (Plato, 2026) . This use of KTF is relevant for two reasons. First, it shows that Plato models can be made operational through automatic transformation into planner-readable artifacts. Second, it exercises central PlanningDomainTransformer capabilities, including the generation of different planning artifacts from semantic source models and the handling of numeric modeling constructs during transformation.

KTF was also used in the evaluation of Pallas. In this case, KTF consumed AAS-based planning models and generated PDDL domain and problem files from them. The Pallas evaluation therefore tested a different source representation than Plato: planning knowledge was not represented as an OWL ontology, but as AAS submodels and submodel elements. KTF also supported transformations between Plato and Pallas where the represented planning information was covered by both models (Pallas, 2026) . This provides evidence that the PlanningDomain-

Transformer is not restricted to one semantic representation, but can act as a transformation component to transform between ontology-based, AAS-based, and planner-oriented artifacts.

For JSOnto, KTF plays a slightly different role. JSOnto is a JSON-based ontology syntax intended to make ontology modeling more modular and human-friendly. KTF operationalizes this syntax by validating JSOnto ontologies and converting them into established ontology serializations such as RDF/XML and Turtle (JSOnto, 2026) . This is important because as JSOnto is a new syntax it is not yet supported by any tooling except for KTF and even KTF's Apache Jena-based PlanningDomainTransformer currently consumes OWL/RDF-based Plato ontologies rather than JSOnto files directly. A Plato ontology modeled in JSOnto can therefore first be validated and transformed into RDF/XML before it is passed to tooling, such as the PlanningDomainTransformer. For the Plato evaluation all the example ontologies were modeled in JSOnto and then transformed to OWL serializations by KTF.

KTF was further used in the [redacted for blind review] project, where Plato, Pallas, and JSOnto were all employed in a broader edge-cloud planning and execution environment. In the project a shared production scenario was examined in which production processes had to be planned and executed across the two research factories SmartFactoryOWL[3] and SmartFactoryKL[4]. This scenario used real production services such as 3D printing, assembly, transport, and quality control, described through both Plato (using JSOnto) and Pallas models. The scenario was also used to evaluate whether semantic models and automatically generated planning domains could support flexible process control under different customer preferences and optimization criteria, including cost, time, and CO_2-related metrics.

For this scenario, a distributed KubeEdge[5]-based edge-cloud infrastructure was set up (KubeEdge is an extension of Kubernetes for making nodes more usable on edge devices). Both KTF and a planning service were deployed on this infrastructure. The workflow first modeled the scenario and its customer preferences using ontology and AAS models. KTF was then used as a service to dynamically generate PDDL planning domains and problems. These generated artifacts were checked using syntactic tools such as PDDL parsers and VAL, and were also inspected manually by domain experts. After plan generation, the resulting plans were again checked automatically and manually. KTF was then used to transform the verified plans into BPMN process models, which were executed by workflow engines in the two smart factories.

No syntactic or semantic errors were found in the PDDL domains generated by KTF during the conducted tests. The plans validated with VAL were consistent with the generated domains and satisfied the specified preconditions and effects of the modeled services. Additionally, it was confirmed that different optimization metrics led to different plausible plans, for example, faster but more emission-intensive plans versus slower but less emission-intensive plans. In constructed cases where customer preferences were incompatible with the available domain, the system generated a valid plan that did not fully satisfy the preference rather than terminating without a solution. Finally, the automatically generated BPMN processes were executed without any runtime errors.

This practical use case provides evidence that KTF can support realistic research workflows and practical deployment scenarios. It covers different kinds of source artifacts, including JSOnto-authored Plato ontologies and AAS-based Pallas models, as well as different target artifacts, including PDDL and BPMN. It also shows that KTF can be deployed as a service in a Kubernetes/KubeEdge environment and used as part of a larger planning and execution pipeline. At the same time, the evidence remains indirect. The reported uses do not constitute an independent stress test of KTF, do not establish complete correctness for all supported inputs, and do not provide a controlled comparison with alternative approaches. They nevertheless show that KTF has been reused across multiple knowledge-engineering artifacts and applied in a non-trivial research project with distributed deployment, semantic modeling, dynamic PDDL generation, plan validation, and BPMN-based execution.

5 DISCUSSION

KTF contributes a practical integration layer for knowledge engineering. Plato, Pallas, and JSOnto each address a specific modeling problem: ontology-based planning knowledge, AAS-based planning knowledge, and JSON-based ontology syntax. KTF makes these artifacts operational by validating, transforming, composing, and exporting them into forms that external tools can use.

The most important architectural lesson is that the framework must be described at two levels. At the framework level, KTF is a collection of tools with

[3]https://www.smartfactory-owl.de/en/

[4]https://smartfactory.de/

[5]https://kubeedge.io/

shared access and deployment infrastructure. At the PlanningDomainTransformer level, the CIP methodology provides a reusable transformation architecture for planning knowledge. Conflating these two levels would obscure the roles of the JSOntoConverter and PlanInterpreter, which are part of KTF but do not themselves follow the same CIP architecture.

KTF's manipulation capabilities also show that knowledge transformation is not merely serialization. Planner-specific generation, action replaying, and numeric discretization are examples of transformations that deliberately change or complete a target artifact according to configuration and target-system constraints. This is useful in practice, but it also means that transformations must be documented carefully. Some transformations may be lossy or target-specific, and not every OWL, AAS, or PDDL construct can be expected to survive every transformation unchanged.

The current evaluation has limitations. It is based on exemplary usages for other research, and representative scenarios in a research project, not on a controlled comparison with alternative tools. The use of KTF for research relating to Plato, Pallas, and JSOnto, as well as the usage in the [redacted for blind review] project, provides initial evidence of practical feasibility, but those usages stem from the same research context as KTF. Further evaluation should include independent users, user satisfaction examination, larger transformation test suites, runtime measurements, and semantic equivalence checks for transformations where such checks are meaningful.

6 CONCLUSION AND FUTURE WORK

This paper presented KTF, the Knowledge Transformation Framework. KTF is an open-source, documented, Dockerized, and mature framework for transforming and manipulating knowledge artifacts. Its tools support semantic planning-domain transformation, ontology syntax conversion and validation, and plan conversion.

The largest contribution of KTF is the PlanningDomainTransformer and its CIP methodology, which separates source-specific consumers, an intermediate planning representation, and target-specific producers. This enables transformations between Plato, Pallas, PDDL, and statistics outputs while also supporting manipulation methods such as planner-specific rewrites, action replaying, and numeric discretization. The framework-level contribution is broader: KTF collects these capabilities into one reusable toolchain that can be called locally, through an API, or from containerized deployments.

Future work should extend the number of supported consumers and producers, document transformation semantics more formally, and add extend the evaluation.

ABBREVIATIONS

The following abbreviations are used in this manuscript:

AAS	Asset Administration Shell
AI	Artificial Intelligence
API	Application Programming Interface
ATL	Atlas Transformation Language
BPMN	Business Process Model and Notation
CIP	Consumer–Intermediary–Producer
GIPO	Graphical Interface for Planning with Objects
HTTP	Hypertext Transfer Protocol
I4.0	Industry 4.0
IoT	Internet of Things
JSON	JavaScript Object Notation
KTF	Knowledge Transformation Framework
LLM	Large Language Model
MIT	Massachusetts Institute of Technology
OWL	Web Ontology Language
PDDL	Planning Domain Definition Language
RDF	Resource Description Framework
REST	Representational State Transfer
SMC	Submodel Collection
SML	Submodel List
UML	Unified Modeling Language
XML	Extensible Markup Language

ACKNOWLEDGEMENTS

[redacted for blind review]

USE OF GENERATIVE AI

ChatGPT (OpenAI, 2026) was used to polish the wording and correct spelling and grammar mistakes for the sections in this paper. Each section was first written by the authors without the use of any AI tools before being passed to the Large Language Model

(LLM) for polishing. All LLM-adjusted texts were carefully checked sentence by sentence for accuracy by the authors. No graphs, figures, code, data, or citations were AI-generated.

REFERENCES

Ammar, S. and Bhiri, M. T. (2018). Automatic planning: From event-b to PDDL. In Abdelwahed, E. H., Bellatreche, L., Benslimane, D., Golfarelli, M., Jean, S., Méry, D., Nakamatsu, K., and Ordonez, C., editors, *New Trends in Model and Data Engineering - MEDI 2018 International Workshops, DETECT, MEDI4SG, IWCFS, REMEDY, Marrakesh, Morocco, October 24-26, 2018, Proceedings*, volume 929 of *Communications in Computer and Information Science*, pages 247–254, Cham Switzerland. Springer.

Beckett, D., Berners-Lee, T., Prud'hommeaux, E., and Carothers, G. (2014). Rdf 1.1 turtle. Technical report, W3C. http://www.w3.org/TR/2014/REC-turtle-20140225/.

Carroll, J. J., Dickinson, I., Dollin, C., Reynolds, D., Seaborne, A., and Wilkinson, K. (2004). Jena: implementing the semantic web recommendations. In Feldman, S. I., Uretsky, M., Najork, M., and Wills, C. E., editors, *Proceedings of the 13th international conference on World Wide Web - Alternate Track Papers & Posters, WWW 2004, New York, NY, USA, May 17-20, 2004*, pages 74–83. ACM.

Chenouard, R., Granvilliers, L., and Soto, R. (2010). Rewriting constraint models with metamodels. *CoRR*, abs/1002.3023.

Fox, M. and Long, D. (2003). Pddl2.1: An extension to pddl for expressing temporal planning domains. *J. Artif. Intell. Res. (JAIR)*, 20:61–124.

Gilchrist, A. (2016). *Industry 4.0: The Industrial Internet of Things*. Apress, USA, 1st edition.

Green, C. (1969). Application of Theorem Proving to Problem Solving. In *1st IJCAI Proc.*, IJCAI'69, page 219–239, San Francisco, CA, USA. Morgan Kaufmann Publishers Inc.

Haslum, P. (2006). *Admissible Heuristics for Automated Planning*. PhD thesis, Linköping University, KPLAB - Knowledge Processing Lab, The Institute of Technology.

Haslum, P., Lipovetzky, N., Magazzeni, D., and Muise, C. (2019). *An Introduction to the Planning Domain Definition Language*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers, California, USA.

Helmert, M. (2009). Concise finite-domain representations for PDDL planning tasks. *Artif. Intell.*, 173(5-6):503–535.

Helmert, M. (2011). The fast downward planning system. *CoRR*, abs/1109.6051.

Horridge, M. and Bechhofer, S. (2011). The OWL API: A java API for OWL ontologies. *Semantic Web*, 2(1):11–21.

Howey, R., Long, D., and Fox, M. (2004). VAL: automatic plan validation, continuous effects and mixed initiative planning using PDDL. In *16th IEEE International Conference on Tools with Artificial Intelligence (ICTAI 2004), 15-17 November 2004, Boca Raton, FL, USA*, pages 294–301. IEEE Computer Society.

Industrial Digital Twin Association and Plattform Industrie 4.0 (2024). *Asset Administration Shell: Reading Guide*. Industrial Digital Twin Association (IDTA). Accessed 16.05.2026.

Jackson, R. C., Balhoff, J. P., Douglass, E., Harris, N. L., Mungall, C. J., and Overton, J. A. (2019). ROBOT: A tool for automating ontology workflows. *BMC Bioinform.*, 20(1):407:1–407:10.

Jouault, F., Allilaire, F., Bézivin, J., and Kurtev, I. (2008). ATL: A model transformation tool. *Sci. Comput. Program.*, 72(1-2):31–39.

JSOnto (2026). JSOnto: A modular json-based syntax for human-friendly ontology engineering. Submitted for review at KEOD 2026. Authors removed for double-blind review since we are referencing our own work. Anonymized preprint available at https://github.com/user140613/Preprints.

Klusch, M., Gerber, A., and Schmidt, M. (2005). Semantic web service composition planning with owls-xplan. *AAAI Fall Symposium - Technical Report*.

Malburg, L., Klein, P., and Bergmann, R. (2023). Converting semantic web services into formal planning domain descriptions to enable manufacturing process planning and scheduling in industry 4.0. *Eng. Appl. Artif. Intell.*, 126:106727.

McCluskey, T. L., Liu, D., and Simpson, R. M. (2003). GIPO II: HTN planning in a tool-supported knowledge engineering environment. In Giunchiglia, E., Muscettola, N., and Nau, D. S., editors, *Proceedings of the Thirteenth International Conference on Automated Planning and Scheduling (ICAPS 2003), June 9-13, 2003, Trento, Italy*, pages 92–101. AAAI.

Micheli, A., Bit-Monnot, A., Röger, G., Scala, E., Valentini, A., Framba, L., Rovetta, A., Trapasso, A., Bonassi, L., Gerevini, A. E., Iocchi, L., Ingrand, F., Köckemann, U., Patrizi, F., Saetti, A., Serina, I., and Stock, S. (2025). Unified planning: Modeling, manipulating and solving AI planning problems in python. *SoftwareX*, 29:102012.

OpenAI (2026). Chatgpt (version 5.5). Large language model.

Pallas (2026). Pallas: Generic asset administration shell-based representation of planning knowledge for automatic PDDL generation. Submitted for review at KEOD 2026. Authors removed for double-blind review since we are referencing our own work. Anonymized preprint available at https://github.com/user140613/Preprints.

Plato (2026). Plato: Generic ontology-based representation of semantic planning knowledge for automatic PDDL generation. Submitted for review at KEOD 2026. Authors removed for double-blind review since we are referencing our own work. Anonymized preprint available at https://github.com/user140613/Preprints.

Vaquero, T. S., Silva, J. R., Tonidandel, F., and Beck, J. C. (2013). itsimple: towards an integrated design system for real planning applications. *Knowl. Eng. Rev.*, 28(2):215–230.

Wickler, G., Chrpa, L., and McCluskey, T. L. (2014). Ontological support for modelling planning knowledge. In Fred, A. L. N., Dietz, J. L. G., Aveiro, D., Liu, K., and Filipe, J., editors, *Knowledge Discovery, Knowledge Engineering and Knowledge Management - 6th International Joint Conference, IC3K 2014, Rome, Italy, October 21-24, 2014, Revised Selected Papers*, Communications in Computer and Information Science, pages 293–312. Springer.